Exception handling

What is **exception**?

- **Exception** is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions.
- These exceptions can occur for various reasons, such as invalid user input, file not found, or division by zero, etc.
- Exception handling in Java is a powerful mechanism to handle runtime errors, ensuring the normal flow of the application even when unexpected conditions occur.
- When an exception occurs within a method, it creates an object known as the exception object. It contains information about the exception, such as the name and description of the exception and the state of the program when the exception occurred.

Exception example

```
public class ExceptionObjectExample {
    public static void main(String[] args) {
        divide(10, 0);  // Attempt to divide by zero
        System.out.println("This line will not be executed.");  // This will not run if an exception occurs
    }

    public static int divide(int numerator, int denominator) {
        return numerator / denominator;  // This line throws an ArithmeticException if denominator is 0
    }
}
```

Output: Exception in thread "main" java.lang.ArithmeticException: / by zero

```
    at ExceptionObjectExample.divide(ExceptionObjectExample.java:10)
    at ExceptionObjectExample.main(ExceptionObjectExample.java:5)
```

Exception handling using **try-catch**

```
public class ExceptionObjectExample {
    public static void main(String[] args) {
        try {
            divide(10, 0);  // Attempt to divide by zero
        }
        catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e);
        }
    }

    public static int divide(int numerator, int denominator) {
        return numerator / denominator;  // This line throws an ArithmeticException if denominator is 0
    }
}
```

Output:

Exception caught: java.lang.ArithmeticException: / by zero

Major reasons why an exception Occurs

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out-of-disk memory)
- Code errors
- Out of bound
- Null reference
- Type mismatch
- Opening an unavailable file
- Database errors
- Arithmetic errors

Exception-Handling Fundamentals

Keyword	Description
try	Program statements that you want to monitor for exceptions are contained within a **try** block. If an exception occurs within the **try** block, it is thrown. we can't use try block alone. The try block must be followed by either catch or finally.
catch	The **catch** block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
throw	System generated exceptions are automatically thrown by the Java run-time system. To manually throw an exception, use the keyword **throw**.
throws	Any exception that is thrown out of a method must be specified as such by a **throws** clause.
finally	Any code that absolutely must be executed after a **try** block completes is put in a **finally** block.

Exception-Handling Fundamentals

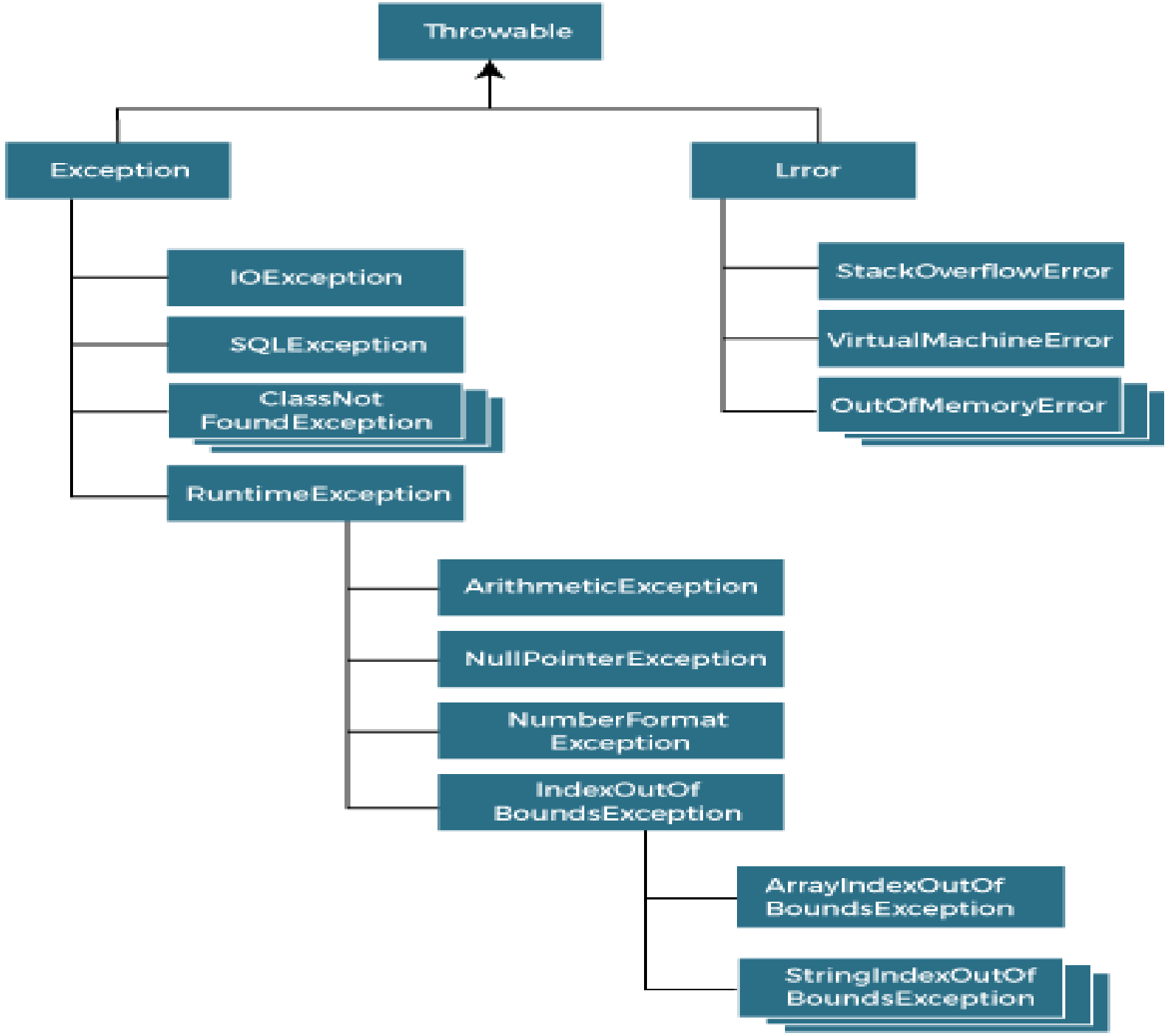
```
try {
    // Code that may throw an exception
} catch (ExceptionType1 e1) {
    // Handle exception of type ExceptionType1
} finally {
    // Code that will always execute (optional)
}
```

Note: Only try block without catch causes compiler time error.

Note: Try with finally block (without catch) still works.

Exception Hierarchy

- All exception types are subclasses of the built-in class **Throwable**.
- **Exception** class is used for exceptional conditions that user programs should catch. This is also the class that you will subclass to create your own custom exception types. Exceptions of this type are automatically defined for the programs that you write and include things such as division by zero and invalid array indexing.
- **Error** class Represents serious problems that a typical application should not attempt to handle (e.g., OutOfMemoryError, StackOverflowError). These are usually JVM issues.


Throwable
Exception
Lrror
IOException
SQLException
ClassNot FoundException
RuntimeException
StackOverflowError
VirtualMachineError
OutOfMemoryError
ArithmeticException
NullPointerException
NumberFormat Exception
IndexOutOf BoundsException
ArrayIndexOutOf BoundsException
StringIndexOutOf BoundsException

Types of exception

- ❖Checked Exception
- ❖Unchecked Exception
- ❖Errors

- Checked Exception: These are the exceptions that are **checked at compile time.** If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using the *throws* keyword.
- **Examples of Checked Exceptions:**
  - **IOException**: Occurs during input/output operations, such as reading or writing files.
  - **SQLException**: Happens during database access or interaction with SQL.
  - **FileNotFoundException**: Thrown when a program attempts to access a file that does not exist.
  - **ClassNotFoundException**: Occurs when a specified class cannot be found, often when dynamically loading classes.
- Any checked exception is a subclass of Exception.

Example (checked exception)

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
public class CheckedExceptionExample {
    public static void main(String[] args) {
        try {
            File file = new File("nonexistent.txt");
            FileReader reader = new FileReader(file);  // This may throw a
FileNotFoundException
        } catch (IOException e) {
            System.out.println("An IOException occurred: " + e.getMessage());
        }
    }
}
```

Unchecked exception

- Unchecked exceptions, also known as **runtime exceptions**, are not checked at compile-time but at runtime.
- These exceptions are typically due to logical errors or programming mistakes and do not need to be declared or caught in the code.
- They are subclasses of RuntimeException.

Examples of Unchecked Exceptions:

•**ArithmeticException**: Occurs when an invalid arithmetic operation is performed, such as dividing by zero.

•**NullPointerException**: Thrown when an application tries to access an object with a null reference.

•**ArrayIndexOutOfBoundsException**: Happens when accessing an array with an illegal index (index out of range).

•**IllegalArgumentException**: Thrown when a method receives an invalid argument.

Example (unchecked exception)

```
public class UncheckedExceptionExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};
        System.out.println(numbers[5]);  // This will throw an arrayIndexOutOfBoundsException
    }
}

// In this case, the code compiles fine, but at runtime, ArrayIndexOutOfBoundsException is thrown due to the illegal array index.
```

Error

- Errors are a special type of Throwable that are not exceptions and generally represent issues that are outside the control of the program or JVM.
- Errors are typically more serious problems, often involving the Java Virtual Machine (JVM) itself, and are generally not meant to be caught or handled by applications.
- Errors are subclasses of java.lang.Error.
- **Examples of Errors:**
  - **OutOfMemoryError**: Thrown when the JVM runs out of memory.
  - **StackOverflowError**: Happens when the call stack overflows, often due to deep or infinite recursion.
  - **VirtualMachineError**: Indicates that the JVM is broken or has encountered a system-level issue.

Example (Error)

```
public class StackOverflowErrorExample {
    public static void main(String[] args) {
        recursiveMethod();  // This will throw a StackOverflowError
    }

    public static void recursiveMethod() {
        recursiveMethod();  // Infinite recursion
    }
}
```

Null pointer exception

```
public class NullPointerExceptionExample {
    public static void main(String[] args) {
        String str = null; // Initializing a String variable to null
        try {
            int length = str.length(); // Attempting to call a method on a null reference
            System.out.println("Length of the string: " + length);
        } catch (NullPointerException e) {
            System.out.println("Error: Null reference encountered.");
            // Additional error handling code can be added here
        }
    }
}
```

Multiple catch Blocks

- You can catch different types of exceptions in multiple catch blocks.
- Specific exceptions should be caught before more general ones.

```
public class MultipleCatchExample {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            int result = numbers[3] / 0;  // This line will throw an ArrayIndexOutOfBoundsException first
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught: Cannot divide by zero.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException caught: Array index is out of bounds.");
        } catch (Exception e) {
            System.out.println("General Exception caught: " + e.getMessage());
        }
        System.out.println("Program continues after exception handling.");
    }
}
```

Important: **Only one catch block is executed**: When an exception is caught, the subsequent catch blocks are skipped.
Order matters: Place more specific exceptions before more general ones to avoid compilation errors.

Example of unreachable cache block

```
public class ExceptionOrderExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will throw ArithmeticException
        } catch (Exception e) {  // General exception
            System.out.println("Caught general exception: " + e.getMessage());
        } catch (ArithmeticException e) { // More specific exception
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        }
    }
}
```

Output:

error: exception ArithmeticException has already been caught

Displaying a Description of an Exception

- The toString() method prints exception information in the format of the Name of the exception: description of the exception.

```
import java.io.*;
class Test
 {
    public static void main (String[] args) {
      int a=5;
      int b=0;
        try{
          System.out.println(a/b);
        }
      catch(ArithmeticException e){
        System.out.println(e.toString());
      }
    }
}
```

- getMessage() : The getMessage() method prints only the description of the exception.

```
System.out.println(e.getMessage());
```

How Does JVM Handle an Exception?

1. Exception Occurrence: When an exception occurs, the JVM creates exception object based on the type of exception (e.g., ArithmeticException, IOException).

- int result = 10 / 0; // ArithmeticException occurs due to division by zero

There might be a list of the methods that had been called to get to the method where an exception occurred. This ordered list of methods is called **Call Stack.**

2. Stack Unwinding: Stack unwinding refers to the JVM's process of unwinding the call stack to find an appropriate exception handler (i.e., a catch block that can handle the exception).

- The JVM starts looking for a matching catch block in the current method, which is the method where the exception occurred.
- If no matching catch block is found, it propagates the exception to the calling method. This continues up the call stack until the exception is either caught or the stack is exhausted.

Example (**Stack Unwinding)** :

```
public class Example {
    public static void main(String[] args) {
        methodA();  // Calling methodA
    }
    public static void methodA() {
        methodB();  // Calling methodB
    }
    public static void methodB() {
        int result = 10 / 0;  // ArithmeticException occurs here
    }
}
```

How the JVM handles this:

- The exception occurs in methodB().
- The JVM searches for a catch block in methodB(). Since there is no catch block in methodB(), the exception propagates to methodA().
- Since methodA() doesn't have a catch block either, the exception propagates to the main() method.
- If there's no catch block in main(), the JVM terminates the program, and the exception is printed on the console.

3. Finding a Matching Catch Block

- The JVM continues to propagate the exception up the call stack, looking for a catch block that can handle the specific type of exception or one of its superclasses.

Example:

```
public class Example {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;  // ArithmeticException occurs here
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
}
}
```

- The exception is caught in the catch (ArithmeticException e) block because it matches the type of the thrown exception (ArithmeticException).
- Once the exception is caught, the catch block is executed, and the program resumes normal execution after the try-catch structure.

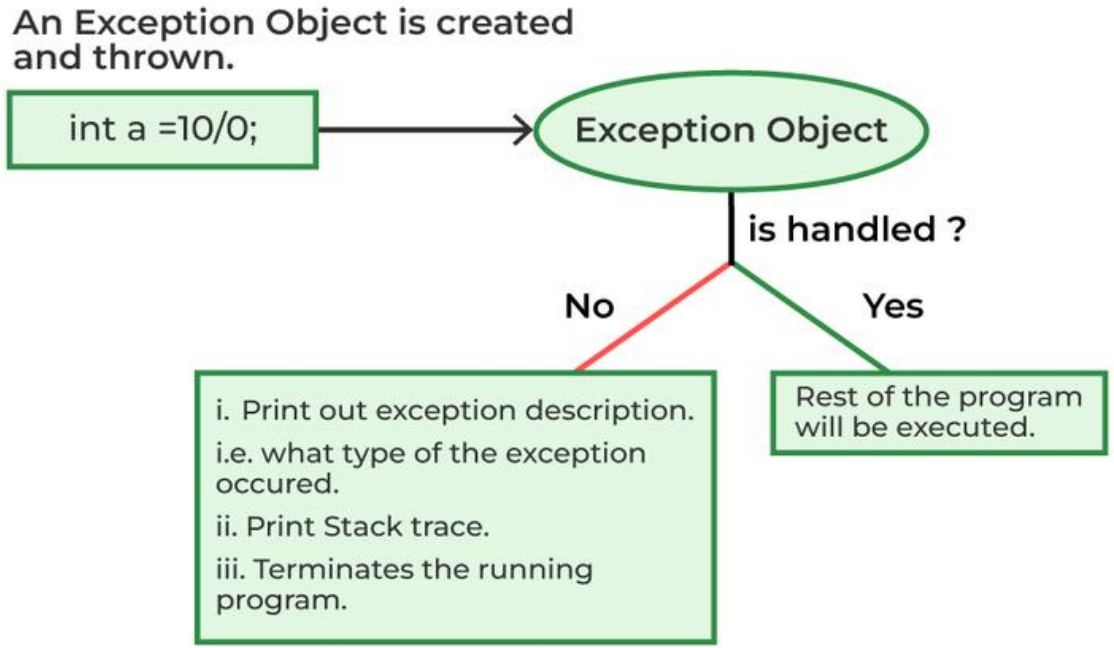
4. **The Finally Block (If Present):**

- After the exception is handled by the catch block, if there is a finally block present, it is executed regardless of whether an exception was thrown or not.
- The finally block is used for cleanup actions like closing file streams, database connections, or releasing resources that were acquired in the try block.
- Example:

```
public class Example {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;  // ArithmeticException occurs here
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } finally {
            System.out.println("This will always execute.");
        }
    }
}
```

5. Propagating Uncaught Exceptions

- If the exception is not caught in any of the catch blocks, it keeps propagating up the call stack.
- If it reaches the main method without being handled, the JVM will:
  - Print the exception's stack trace to the console (which helps in debugging).
  - Terminate the program (the program ends after the exception).



Nested try block

```
public class NestedTryExample {
    public static void main(String[] args) {
        // Outer try block
        try {
            int result = 10 / 0;
            System.out.println("Result: " + result);
            // Inner try block
            try {
                int[] numbers = {1, 2, 3};
                System.out.println(numbers[5]);
            }
             catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Caught ArrayIndexOutOfBoundsException in inner try: " + e.getMessage());
            }
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException in outer try: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed after outer try-catch");
        }
    }
}
```

throw

- It is used to explicitly **throw an exception** in a program.
- Unlike try-catch, which is used to handle exceptions, throw is used to generate an exception.
  - throw *ThrowableInstance*;
  - ThrowableInstance must be an object of type Throwable or a subclass of Throwable.
- The throw keyword is often used to enforce specific conditions within methods or custom validation.

Example (throw)

```
public class NullPointerExample {
    public static void main(String[] args) {
        try {
            printLength(null);  // Passing null to trigger the exception
        } catch (NullPointerException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
    // Method that throws NullPointerException if the input string is null
    public static void printLength(String str) {
        if (str == null) {
             NullPointerException obj=new NullPointerException("The input string cannot be
null.");
            throw obj;}
        System.out.println("Length of the string: " + str.length());
    }}
```

Example (throw): custom validation

```
public class AgeCheck {
    public static void main(String[] args) {
        try {
            register(16);  // Trying to register with age 16
        } catch (IllegalArgumentException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }

    // Method to check age and throw an exception if age is below 18
    public static void register(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or older to register.");
        }
        System.out.println("Registration successful!");
    }
}
```

throws

- **throws** keyword is used in a method signature to declare that a method can potentially throw specific exceptions.
- It does not handle the exception but informs the calling method that it might need to handle or further declare the exception.
- This is commonly used with **checked exceptions**, which are exceptions that must be either caught or declared to be thrown.
  - **Syntax**:

```
public void someMethod() throws IOException, SQLException {
    // Code that may throw IOException or SQLException
}
```

```
// This program contains an error and will not compile.
class ThrowsDemo {
  static void throwOne() {
    System.out.println("Inside throwOne.");
    throw new IllegalAccessException("demo");
  }
  public static void main(String args[]) {
    throwOne();
  }
}
```

To make this example program compile, you need to make two changes.

- First, you need to declare that **throwOne( )** throws **IllegalAccessException**.
- Second, **main( )** must define a **try / catch** statement that catches this exception.

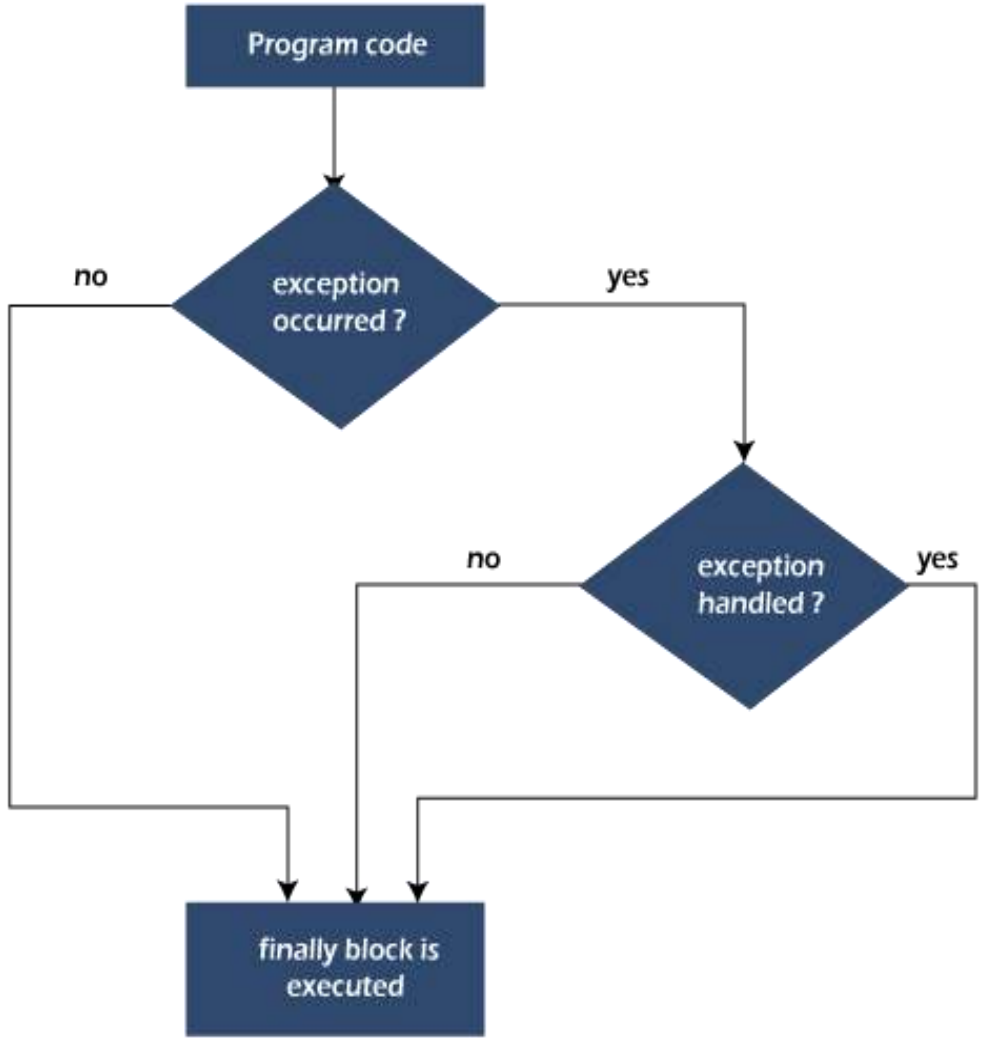
```
// This is now correct.
class ThrowsDemo {
  static void throwOne() throws IllegalAccessException {
    System.out.println("Inside throwOne.");
    throw new IllegalAccessException("demo");
  }
  public static void main(String args[]) {
    try {
      throwOne();
    } catch (IllegalAccessException e) {
      System.out.println("Caught " + e);
    }
  }
}
```

Output:

inside throwOne
caught java.lang.IllegalAccessException: demo

finally

- finally block is an optional part of exception handling that is used to execute important code regardless of whether an exception occurs or not.
- The finally block is always associated with a try block, and it executes after the try block and any catch blocks, even if an exception is thrown.
- This is particularly useful for cleaning up resources such as closing files, releasing database connections, or freeing up other system resources.



Example 1: Exception is Handled

```
public class FinallyExampleHandled {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will throw ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed - Exception handled case.");
        }

        System.out.println("Program continues after try-catch-finally.");
    }
}
```

Output:

Exception caught: / by zero
Finally block executed - Exception handled case.
Program continues after try-catch-finally.

Example 2: Exception is not handled

```
public class FinallyExampleUnhandled {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]); // This will throw ArrayIndexOutOfBoundsException
        } finally {
            System.out.println("Finally block executed - Exception unhandled case.");
        }
        System.out.println("This line will not execute due to unhandled exception.");
    }
}
```

Output:

```
Finally block executed - Exception unhandled case.
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
    at FinallyExampleUnhandled.main(FinallyExampleUnhandled.java:5)
```

Example 3: Exception doesn’t occur

```
public class FinallyExampleNoException {
    public static void main(String[] args) {
        try {
            int result = 10 / 2; // No exception here (10 / 2 is 5)
            System.out.println("Result: " + result);
        } finally {
            System.out.println("Finally block executed - No exception occurred.");
        }

        System.out.println("Program continues after try-finally.");
    }
}
```

Output:

Result: 5

Finally block executed - No exception occurred.

Program continues after try-finally.

Create your own exception (user-defined exception)

- In Java, you can create your own custom exceptions to handle specific error scenarios that aren’t covered by Java’s built-in exceptions.
- This is useful when you want to provide more meaningful error messages or enforce specific application rules.
- Creating custom exceptions involves subclassing the Exception class (or one of its subclasses) and optionally adding custom fields and methods.
- **Steps to Create a Custom Exception**

  1.Define a Class: Define a new class that extends Exception (checked exception) or RuntimeException (unchecked exception), depending on whether you want your exception to be checked or unchecked.
  2.Constructor(s): Add constructors to initialize the exception. Commonly, custom exceptions have constructors that accept a message and optionally a cause (another throwable).
  3.Additional Methods (Optional): Add any additional methods or fields that might help provide more details about the exception.

Example 1: Creating a Custom Checked Exception

- Create a custom checked exception called InvalidAgeException, which is thrown when an invalid age is provided (for example, a negative age or an age greater than 150).

```
class InvalidAgeException extends Exception {          // Define the custom exception
public InvalidAgeException(String message) {           // Constructor that accepts a custom message
      super(message);
   }
}
public class CustomExceptionExample {                  // Using the custom exception
   public static void main(String[] args) {
      try {
         validateAge(-5); // This will throw InvalidAgeException
      } catch (InvalidAgeException e) {
         System.out.println("Caught Exception: " + e.getMessage());
      }
   }
public static void validateAge(int age) throws InvalidAgeException {          // Method to validate age
      if (age < 0 || age > 150) {
         throw new InvalidAgeException("Invalid age: " + age);
      }
      System.out.println("Age is valid: " + age);
   }
}
```

Example 2: Creating a Custom Unchecked Exception

- To create an unchecked exception, extend RuntimeException instead of Exception. This way, the compiler won't require you to handle or declare the exception.

```
class NegativeBalanceException extends RuntimeException {          // Define the custom unchecked exception
    public NegativeBalanceException(String message) {
        super(message);
    }
}
public class CustomUncheckedExceptionExample  {                     // Using the custom unchecked exception
    public static void main(String[] args) {
        withdraw(100, 150); // This will throw NegativeBalanceException
    }
public static void withdraw(int balance, int amount) {              // Method that checks for a valid balance
        if (amount > balance) {
            throw new NegativeBalanceException("Insufficient balance for withdrawal: balance is " + balance);
        }
        System.out.println("Withdrawal successful. Remaining balance: " + (balance - amount));
    }
}
```

Redirecting Exceptions

- In Java, **redirecting exceptions** refers to the process of **catching an exception in one block and then throwing it again (or another exception) to be handled elsewhere**.
- This technique is useful when you want to add custom handling, such as logging or wrapping an exception into a more meaningful one, without fully resolving the issue within the current scope.
- There are two main ways to redirect exceptions in Java:
  - Re-throwing the Same Exception: Simply catch the exception and throw it again.
  - Throwing a Different Exception: Catch one exception and throw a different exception, optionally including the original exception as the cause.

Example 1: Re-throwing the Same Exception

```
import java.io.FileReader;
import java.io.IOException;
public class ReThrowExample {
    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("Exception caught in main: " + e.getMessage());
        }
    }
    public static void readFile() throws IOException {
        try {
            FileReader reader = new FileReader("non_existent_file.txt");
        } catch (IOException e) {
            System.out.println("Logging exception: " + e.getMessage());
            throw e; // Re-throwing the same exception
        }
    }
}
```

Example 2

```
class RethrowExample {
    public static void processData() throws Exception {
        try {
            // Simulate an error
            throw new ArithmeticException("Division by zero");
        } catch (ArithmeticException e) {
            // Log or perform some cleanup
            System.out.println("Logging exception: " + e.getMessage());
            // Rethrow the same exception
            throw e;
        }
    }
}
public class Main {
    public static void main(String[] args) {
        try {
            RethrowExample.processData();
        } catch (Exception e) {
            System.out.println("Exception caught in main: " + e.getMessage());
        }
    }
}
```

Example 2: Redirecting the runtime Exception

```
public class RuntimeExceptionRedirect {
    public static void main(String[] args) {
        try {
            calculate();
        } catch (ArithmeticException e) {
            System.out.println("Caught exception in main: " + e.getMessage());
        }
    }
    public static void calculate() {
        try {
            int result = 10 / 0; // This will cause ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught, re-throwing...");
            throw e; // Re-throwing the same exception
        }
    }
}
```

Example 3: Wrapping and Throwing a Different Exception

- You can catch one type of exception and wrap it in a custom exception to provide more context.

```
class CustomException extends Exception {
    public CustomException(String message, Throwable
cause) {
        super(message, cause);
    }
}
public class WrapExceptionExample {
    public static void main(String[] args) {
        try {
            processFile();
        } catch (CustomException e) {
            System.out.println("Caught CustomException: "
+ e.getMessage());
            System.out.println("Original cause: " +
e.getCause());
        }
    }
    public static void processFile() throws
CustomException {
        try {
            FileReader reader = new
FileReader("non_existent_file.txt");
        } catch (IOException e) {
            throw new CustomException("File processing
failed", e); // Wrap the IOException
        }
    }
}
```

Java Build-in exception classes

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalCallerException	A method cannot be legally executed by the calling code.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
LayerInstantiationException	A module layer cannot be created.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Java's Unchecked **RuntimeException** Subclasses Defined in **java.lang**

Java Build-in exception classes

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions.

Java's Checked Exceptions Defined in **java.lang**